

Denoising Programming Knowledge Tracing with a Code Graph-based Tuning Adaptor

Weibo Gao

State Key Laboratory of Cognitive
Intelligence, University of Science and
Technology of China
Hefei, China
weibogao@mail.ustc.edu.cn

Qi Liu*

State Key Laboratory of Cognitive
Intelligence, University of Science and
Technology of China
Institute of Artificial Intelligence,
Hefei Comprehensive National
Science Center
Hefei, China
qiliuql@ustc.edu.cn

Rui Li

State Key Laboratory of Cognitive
Intelligence, University of Science and
Technology of China
Hefei, China
ruili2000@mail.ustc.edu.cn

Yuze Zhao

State Key Laboratory of Cognitive
Intelligence, University of Science and
Technology of China
Hefei, China
yuzezhao@mail.ustc.edu.cn

Hao Wang

State Key Laboratory of Cognitive
Intelligence, University of Science and
Technology of China
Hefei, China
wanghao3@ustc.edu.cn

Linan Yue

State Key Laboratory of Cognitive
Intelligence, University of Science and
Technology of China
Hefei, China
lunyue@mail.ustc.edu.cn

Fangzhou Yao

State Key Laboratory of Cognitive
Intelligence, University of Science and
Technology of China
Hefei, China
fangzhouyao@mail.ustc.edu.cn

Zheng Zhang

State Key Laboratory of Cognitive
Intelligence, University of Science and
Technology of China
Hefei, China
zhangzheng@mail.ustc.edu.cn

Abstract

Programming Knowledge Tracking (PKT) aims to dynamically diagnose learners' mastery levels of programming knowledge based on their coding activities, facilitating more effective and personalized programming education. However, current PKT studies primarily focus on the implicit relationship between code content and knowledge assessment, often overlooking two types of noise signals in long-term programming activities: unwanted signals from unrelated submissions and weak signals from minor modifications. This practical challenge significantly limits model performance and application. To address this issue, we propose **Coda**, a Code graph-based tuning adaptor designed to enhance existing PKT models by identifying and mitigating the impact of noise. Specifically, Coda first transforms the loose code sequences submitted by each learner into a compact code graph. By leveraging this code graph,

unwanted signals can be identified from a semantic similarity perspective. We then apply a cluster-aware GCN to the code graph, which improves the discrimination of weak signals and enables their clustering for identification. Finally, a lightweight yet effective adaptor is incorporated into the PKT task through optimization with two noise feature-based constraints and a navigational regularization term, to correct knowledge states affected by noise. It is worth mentioning that the Coda framework is model-agnostic and can be adapted to most existing PKT solutions. Extensive experimental results on four real-world datasets demonstrate that Coda effectively performs the PKT task in the presence of noisy programming records, outperforming typical baselines.

CCS Concepts

• Applied computing → E-learning.

Keywords

User Modeling, Programming Knowledge Tracing, Learning Performance Prediction, Intelligent Education, Data Denoising

ACM Reference Format:

Weibo Gao, Qi Liu, Rui Li, Yuze Zhao, Hao Wang, Linan Yue, Fangzhou Yao, and Zheng Zhang. 2025. Denoising Programming Knowledge Tracing with a Code Graph-based Tuning Adaptor. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.1 (KDD '25)*, August 3–7, 2025, Toronto, ON, Canada. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3690624.3709172>

* Corresponding Author.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
KDD '25, Toronto, ON, Canada

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-1245-6/25/08
<https://doi.org/10.1145/3690624.3709172>

1 Introduction

With the advent of online programming platforms like *LeetCode.com*, it has become increasingly popular for students and other learners to use these websites for coding training [10, 27, 47]. As data on online programming activities accumulate, understanding the programming training process is crucial for enhancing online learning services, such as question recommendation [43, 51].

Human programming practice generally involves an iterative process of code submission and modification to solve tasks. Learners submit code and receive feedback, such as error messages like *Compile Error*, or positive feedback like *Accepted*. Based on the specific feedback, they revise their previous submissions. Figure 1 illustrates this process, where a learner attempts to solve each question (e.g., q_1) through multiple submissions. During this process, learners develop an internal understanding of the question, or technically speaking, a hidden computational knowledge state, representing their learning progress. As learners refine their code based on feedback, their knowledge states evolve, reflecting their learning journey. To better understand programming learning, it is essential to model and trace the evolution of these invisible knowledge states, which is the focus of Programming Knowledge Tracing[54].

Programming Knowledge Tracing (PKT) involves dynamically evaluating learners' programming knowledge states based on their coding activities. These knowledge states refer to learners' proficiency in various programming knowledge concepts such as *Array* and *Function*. PKT extends traditional knowledge tracing (KT) techniques [35, 38], which are designed for non-programming general-purpose learning environments [21], by incorporating code-related features specific to programming environments. Its tracing results offer significant insights into students' coding abilities [18, 25, 54]. Since knowledge states cannot be directly observed, most attempts model PKT as a next-performance prediction problem, usually using recurrent neural networks (RNNs) to summarize programming sequences [47]. These models mine code features to infer learner knowledge states from their codes[21, 23, 36], establishing implicit relationships between code and knowledge [54], and achieving high predictive performance. However, these models are limited by the numerous noisy submissions in real-world programming data, which stem from the open-ended nature of programming platforms.

Through examining online programming environments [29, 55], we identify two primary categories of prevalent noisy signals during programming:

- **Unwanted signals from unrelated submissions.** In the majority of learners' programming practice records, there are commonly codes that are unrelated to the solved questions. Typical examples include: (1) Novice programmers are unfamiliar with the web-based coding environment and use simple but irrelevant code snippets (e.g., `print('Welcome to KDD 2025')`) to test the use of the online coding environment. (2) Learners may carelessly submit codes to inappropriate questions. For instance, a learner might mistakenly submit the solution for the question "Add Two Numbers" to a similar question "Add Binary" when both submission pages are open simultaneously in their web browser. These records do not reflect the learner's knowledge and introduce unwanted signals in their programming history, complicating the accurate modeling of true knowledge states.

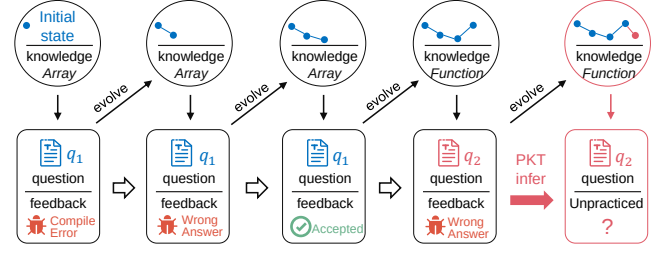


Figure 1: The example of the programming practice process, where a PKT model is used to infer future performance.

- **Weak signals from minor modifications.** Solving a programming problem typically involves multiple submissions, where many submissions are minor modifications of previous ones based on feedback. After several failed attempts, learners may submit an entirely new code. We argue that learners' knowledge states are likely to remain unchanged for minor modification submissions, as the modifications are not significant improvements. While these minor changes may be correct or meaningful for humans, they are redundant to PKT models. These weak signals can mislead the PKT model if used directly in training, leading to overestimation or underestimation of the learner's knowledge state.

Note: The noisy signals described above provide two typical insights for our preliminary exploration. Many other types of noise exist in programming data, which we plan to explore in future research. Additionally, although these so-called noisy code contents may be meaningful from a human perspective, they could still represent irrelevant or redundant signals for current PKT technology, so we classify them as noise.

In this paper, our goal is to mitigate the impact of these noisy signals to enhance existing PKT models, which presents two primary challenges: (1) Accurately identifying noise signals is challenging due to the absence of noise labels or noise annotation criteria. The available clues are the submitted codes, feedback, and question information. (2) Designing an effective and lightweight method to use noise identification to denoise existing PKT solutions is significant, due to the computational complexity of PKT tasks.

To address these challenges, we propose **Coda**, a **code** graph-based tuning adaptor designed to enhance existing PKT models through denoising. While mature denoising methods exist in other fields [12], this is the first attempt to address denoising within PKT. Our model comprises two key stages: the *noisy signal identification stage* and the *adaptive PKT stage*. In the noisy signal identification stage, we construct a code graph based on programming sequences to explore semantic relations between codes. Using this code graph, we assess unwanted signals from unrelated codes by examining semantic similarities between codes and their correlations with questions. To distinguish weak signals from submissions, we employ a cluster-aware Graph Convolution Network (GCN) over the code graph, which improves the discrimination of weak signals and enables their clustering for identification. In the adaptive PKT stage, a tuning adaptor facilitates the adaptation of the PKT task to the noisy signals by tuning a trained PKT backbone model, ensuring computational efficiency. Two noise code feature-based constraints and a navigational regularization term with theoretical derivation

guide the tuning process, ensuring effective adaptation of PKT. It is worth mentioning that the Coda framework is model-agnostic and can be adapted to most existing PKT models. Extensive experiments on four real-world datasets demonstrate that Coda effectively performs the PKT task in the presence of noisy programming records, surpassing typical baselines.

2 Preliminaries

2.1 Problem Definition

In a programming scenario, let $\mathcal{U}$, $\mathcal{Q}$, and $\mathcal{C}$ denote the sets of N learners, M questions, and K knowledge concepts, respectively. Each question typically tests one primary knowledge concept such as *Array*. Each learner’s practice activities consist of a sequence of submitted codes and corresponding feedback. For learner u at submission step t , he/she attempts to solve a question $q_{u,t} \in \mathcal{Q}$, drawn from a knowledge concept $c_{u,t} \in \mathcal{C}$, by writing and submitting code $x_{u,t}$, and receives feedback $r_{u,t}$, such as *Wrong Answer* or *Compile Error*. Feedback messages also reflect the learner’s programming performance. Existing PKT models typically represent $r_{u,t}$ as a binary value: $r_{u,t} = 1$ indicates that learner u received an correct result (i.e., solved the question $q_{u,t}$ with *Accepted* feedback), while $r_{u,t} = 0$ otherwise. Thus, each learner u has a sequence of programming records:

$$l_u = (q_{u,1}, c_{u,1}, x_{u,1}, r_{u,1}), \dots, (q_{u,T_u}, c_{u,T_u}, x_{u,T_u}, r_{u,T_u}), \quad (1)$$

where T_u denotes the length of learner u ’s programming sequence.

The evolution of the learner’s knowledge proficiency during the programming process is hard to record and quantify explicitly. Therefore, most previous approaches model PKT as a next-attempt performance prediction problem. The optimization objective is to minimize the cross-entropy loss between the predicted performance $\hat{r}_{u,t+1}$ (i.e., whether the learner will answer correctly on the next attempt) and the actual performance $r_{u,t+1}$ at attempt t [47]:

$$\mathcal{L}_{u,t}^{pkt} = -r_{u,t+1} \log \hat{r}_{u,t+1} - (1 - r_{u,t+1}) \log (1 - \hat{r}_{u,t+1}). \quad (2)$$

By jointly training on performance predictions using all programming data, the knowledge state of learners with respect to each question-related knowledge concept can be optimized.

In summary, we formally define the problem of Programming Knowledge Tracing (PKT) as follows:

Definition 1 (Programming Knowledge Tracing). Given the prior programming records of a learner u up to and including submission step t , represented as a sequence $\{(q_{u,1}, c_{u,1}, x_{u,1}, r_{u,1}), \dots, (q_{u,t}, c_{u,t}, x_{u,t}, r_{u,t})\}$, the goal is to: (1) estimate the internal programming knowledge state $h_{u,t}$ at attempt t of this learner; (2) predict the performance $\hat{r}_{t+1}$ on the next question $q_{u,t+1}$.

2.2 PKT Backbone Model

Our Coda framework needs to tune a trained PKT model (as the backbone model), such as DKT+ [39], so this part introduces the general structure of the existing PKT model to serve as the basis for the § 3 framework introduction.

Almost all of previous PKT models employ a sequential way, e.g., RNN, to recurrently model each coding submission of the learner. Each recurrent unit typically consists of three basic components that work conjointly processing each coding submission once, until

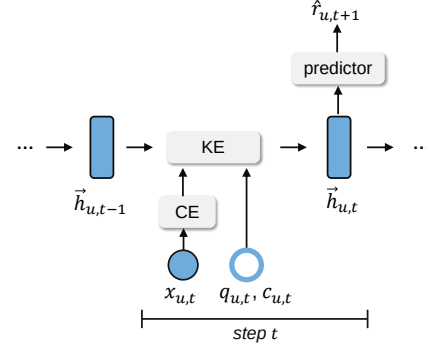


Figure 2: The basic architecture of the PKT model.

the end of the programming sequence: the **code encoder (CE)**, the **knowledge estimation (KE)** and the **predictor**, as illustrated in Figure 2. At each attempt of the programming practice sequence (index by t), (1) **CE** first extracts and represents the code feature $\tilde{x}_{u,t}$ from a learner u ’s submission $x_{u,t}$ at attempt t , i.e., $\tilde{x}_{u,t} = \text{CE}(x_{u,t})$. Existing CE modules either extract semantic features regarding codes as a special language using NLP techniques [5, 54] or first transform the code to a graph (e.g., abstract syntax tree [2, 21]) and then encode structural dependencies using graph modeling methods. (2) The **KE** component then estimates the knowledge state $\tilde{h}_{u,t}$ for learner u via the state $\tilde{h}_{u,t-1}$ from the last attempt and the current submission record, i.e., $\tilde{h}_{u,t+1} = \text{KE}(\tilde{h}_{u,t-1}, \tilde{q}_{u,t}, \tilde{c}_{u,t}, \tilde{x}_{u,t}, r_{u,t})$. The embeddings $\tilde{q}_{u,t}$ and $\tilde{c}_{u,t}$ are usually obtained from their textual contents or learned through joint training [54]. (3) The **predictor** finally predicts the learner’s performance $\hat{r}_{u,t+1}$ to the next question $q_{u,t+1}$ based on the current knowledge state $\tilde{h}_{u,t}$.

In our paper, we will not focus on the details of the pkt backbone model. The training and test details of a PKT backbone model are given in Appendix C. Given a well-trained PKT backbone model, our Coda and bindings with it through its three key components described above.

3 The Coda Framework

Coda is a general and model-agnostic framework designed to help existing PKT models recognize noisy programming submissions and correct compromised knowledge states. It can be integrated with PKT models in various ways, such as tuning or end-to-end joint training. For the sake of computational efficiency (as elaborated in § 1), we adopt the tuning approach. Specifically, we first train a conventional PKT model (e.g., DKT+ [39]) as the backbone. Then, we incorporate Coda into the backbone model and only train Coda’s parameters while keeping the PKT backbone fixed. Figure 3 illustrates our proposed Coda framework comprising two stages: the *noisy signal identification stage* (§ 3.1) and the *adaptive PKT stage* (§ 3.2). Assuming we have obtained a trained PKT backbone model, we will elaborate on each stage of Coda in detail.

3.1 Noisy Signal Identification Stage

Identifying unwanted and weak noise signals accurately is extremely difficult from programming sequences because we lack a clear judging criterion or available labels regarding noisy submissions. Through the discussion of noisy submissions in § 1, we

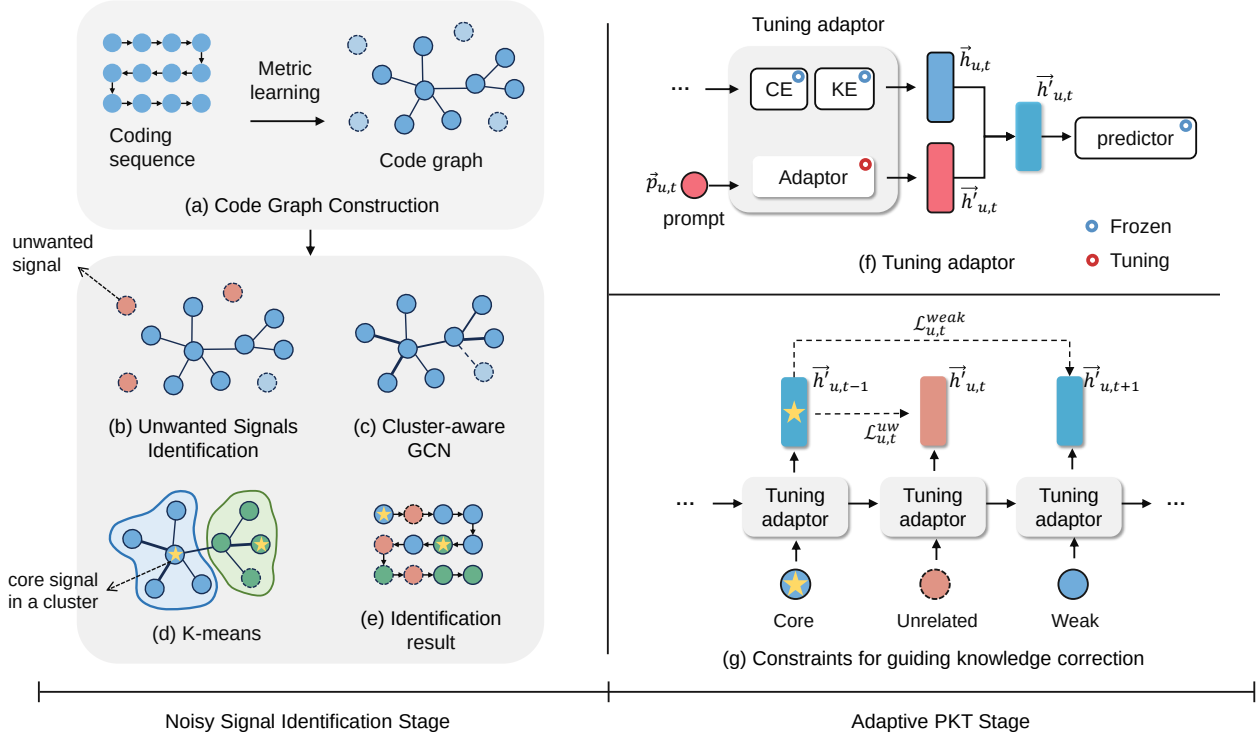


Figure 3: The overall architecture of Coda. Left shows the stage for noisy signal identification, where (b) aims to find unwanted signals, and (c) and (d) perform weak signal fusion. Right shows the adaptive PKT stage with the tuning of Coda.

argue that noisy submissions and regular submissions from a specific learner exhibit distinct semantic characteristics: *unwanted submissions differ significantly from normal submissions in terms of code contents, while weak submissions tend to resemble a certain core submission*. This stage conducts noise identification by exploiting semantic relations between codes submitted by a specific learner.

3.1.1 Code Graph Construction. To fully exploit the semantic relations between codes submitted by a specific learner, we convert the loose submitted code sequence of each learner into a tight code graph. The nodes on the graph represent the submitted code at each attempt, and the edges indicate the existence of semantic similarities between a pair of nodes. Intuitively, for a specific learner, unwanted code nodes may have very few edges even within a multi-hop range, and they might even be isolated. Meanwhile, a group of weak submission nodes may cluster around a central core node. Therefore, if we can construct an effective code graph for the learner’s submitted code sequence, it becomes easier to identify and model noisy signals.

Inspired by previous works [3, 19], we propose an adaptive way based on metric learning to automatically construct code graph structures for each programming practice sequence to explore the distribution of different coding signals. Technically speaking, this module aims to construct an undirected code graph $G_u = \{V_u, A_u\}$ where $A_u \in \mathbb{R}^{T_u \times T_u}$ is the learnable adjacency matrix, and V_u is the submitted code set of a learner. Each node $v_{u,i} \in V_u$ corresponds to the code $x_{u,i}$ at the attempting step i within the programming sequence. The node embedding vector $\vec{x}_{u,i} \in \mathbb{R}^d$ is obtained from

the CE (defined in § 2.2) in a refined PKT backbone model. Various graph construction techniques have been proposed to learn graph structure [15, 22, 44]. Since the focus of this paper is not to devise more sophisticated techniques for graph learning, we use a simple yet effective similarity metric learning method [3, 40, 52] to transform the graph learning problem into node similarity evaluations. Specifically, we first calculate a metric matrix $\mathbf{M}_u \in \mathbb{R}^{T_u \times T_u}$. Each of its element $M_{u,<i,j>}$ is calculated as:

$$M_{u,<i,j>} = \text{sim}(\mathbf{W} \odot \vec{x}_{u,i}, \mathbf{W} \odot \vec{x}_{u,j}), \quad (3)$$

where $\text{sim}(\cdot, \cdot)$ is a metric function to calculate the similarity metric between code node $v_{u,i}$ and $v_{u,j}$ corresponding to the i -th and j -th submissions, respectively, in u ’s programming sequence. It is flexible and can be implemented by various forms. $\mathbf{W} \in \mathbb{R}^d$ is a trainable vector to adaptively highlight different dimensions of the code embeddings $\vec{x}_{u,i}$ and $\vec{x}_{u,j}$. $\odot$ denotes the Hadamard product [16]. We implement $\text{sim}(\cdot, \cdot)$ using a weighted cosine similarity with the consideration of balancing expressiveness and complexity [4, 24].

After obtaining the metric matrix $\mathbf{M}_u$, it is necessary to normalize it to ensure non-negativity, considering that the cosine values $M_{u,<i,j>}$ fall within the range of $[-1, 1]$. Simply normalizing it without additional constraints may not regulate the graph sparsity, potentially resulting in a fully connected adjacency matrix [3, 49]. This may reduce the discrimination of relations between different code pairs, which is harmful to the observation of core and weak submissions, as well as limit the occurrence of isolated nodes, which is harmful to the observation of unwanted submissions. To better focus on the vital connections and disconnect the unwanted

submissions, we adopt a ranking strategy [3] to control the sparsity of the graph. Specifically, we mask off those elements in $\mathbf{M}_u$ that are smaller than a hyper-parameter by setting them to zero, which is obtained by ranking the metric value in $\mathbf{M}_u$ to generate an informative adjacency matrix A_u , as follows:

$$A_{u,<i,j>} = \begin{cases} 1, & M_{u,<i,j>} \geq \text{Rank}_\epsilon(\mathbf{M}_u) \\ 0, & \text{otherwise} \end{cases} \quad (4)$$

where $\text{Rank}_\epsilon(\mathbf{M}_u)$ returns the value of the ϵ -th largest value in the metric matrix $\mathbf{M}_u$ to control the sparsity of the code graph. Note that the structure of the code graph is learnable and changes continuously with the update of $\mathbf{W}$. The construction method is general, easy to implement, and able to perfectly cope with inductive learning (with new codes during test). Based on the code graph, we then conduct noise signal identification by exploiting similarity relations between nodes.

3.1.2 Unwanted Signals Identification. This part aims to identify noisy signals originating from unrelated submissions. Intuitively, unrelated codes correlate weakly with other nodes within the code graph G_u of a specific learner u . Therefore, they are most likely to be disconnected from other nodes within G_u and become isolated nodes during the construction process.

However, it is difficult to determine unwanted submissions solely by relying on isolated nodes, as these may also represent novel or unique problem-solving approaches. To enhance the confidence in identifying unwanted nodes, we introduce an auxiliary view to exploit correlations between isolated code nodes and question-related solutions. The solutions refer to the correct submissions corresponding to the specific question. We argue that the correct solution must correspond to the practiced questions; otherwise, it will not pass the online judge. If a code node is similar to a certain solution, it is likely to be a useful and related submission signal for the current question. Notably, codes with consistently low similarity to each solution related to the practiced question are not necessarily irrelevant to the question. These submissions may be relatively short and simple code snapshots that are question-related but require multiple modifications to solve the question correctly. At a glance, this idea cannot directly determine irrelevant signals. However, we can determine noisy signals by identifying those that are both isolated and question-uncorrelated simultaneously. Our basic assumption is that a submission that is both isolated and irrelevant to each solution is less relevant to the rest of the learner's programming sequence and the current question, as empirically verified by previous studies [23].

Based on the above analysis, for each practiced question $q_{u,i}$ by u , we collect all the correct submissions (codes with the feedback *Accepted*) from the entire training data corresponding to it as its solution set $S_{q_{u,i}}$. We then calculate the similarity between the embeddings of isolated code $x_{u,i}$ and each solution code $x_{u,j'}$ as:

$$S_i = \{\{s_{i,j'}\} | j' \in S_{q_{u,i}}, s_{i,j'} = \text{sim}(\mathbf{W} \odot \vec{x}_{u,i}, \mathbf{W} \odot \vec{x}_{u,j'})\}. \quad (5)$$

The similarity evaluation function $\text{sim}(\cdot, \cdot)$ is implemented using cosine similarity as Eq. (3). If the mean w.r.t. the distribution of S_i is less than its median, we empirically consider the x_i as an irrelevant submission.

After identifying the unwanted nodes, we remove them from the node set V_u . Each of the remaining isolated but question-related nodes needs to be added to the graph by linking the most similar node with it via cosine similarity score for further calculation. This process may result in multiple disjoint subgraphs (each containing at least two nodes) rather than necessarily forming a connected graph. In both cases, we uniformly denote them as G_u for simplicity, as it will not affect subsequent calculations.

3.1.3 Cluster-aware Weak Signal Fusion. This section aims to distinguish strong signals from core code submissions and weak signals from minor modifications. In the context of programming, weak signals primarily stem from previous submissions with minor adjustments based on feedback information. These weak signals and their corresponding core submissions are more likely to be grouped into a cluster via their representations. Motivated by this idea, we first apply an alternative cluster-aware Graph Convolution Network (GCN) on the code graph to improve the discrimination among code nodes by gathering weak signals to strong ones.

The input of the cluster-aware GCN is a node embedding matrix $\{\vec{x}_{u,1}, \vec{x}_{u,2}, \dots, \vec{x}_{u,n}\}$, where each $\vec{x}_{u,t} \in \mathbb{R}^d$ is the code $x_{u,t}$'s representation and $n < T_u$ is the number of nodes on the graph G_u after removing unwanted nodes (i.e., the length of the learner programming sequence on a certain question besides unrelated submissions). The output is a new node embedding matrix $\{\vec{x}'_{u,1}, \vec{x}'_{u,2}, \dots, \vec{x}'_{u,n}\}$, where $\vec{x}'_{u,i} \in \mathbb{R}^{d'}$ with potentially different dimension d' . We iteratively conduct GCN **Aggregate**($\cdot$) with $L > 0$ times as:

$$\begin{aligned} \vec{x}'_{u,i} &= \vec{x}_{u,i}^{(L)} + \vec{x}_{u,i}, \\ \vec{x}_{u,i}^{(l)} &= \sigma \left(\mathbf{W}_a \cdot \text{Aggregate} \left(\alpha_{i,j} \cdot \vec{x}_{u,j}^{(l-1)}; j \in \mathcal{N}_i \right) \right), l = 1 \sim L. \end{aligned} \quad (6)$$

The above Eq. (6) shows how a code node $v_{u,i}$ gathers the semantic signals from its neighboring set $\mathcal{N}_{u,i}$ as well its original embedding $\vec{x}_{u,i}$. $\vec{x}_{u,i}^{(0)}$ is defined as $\vec{x}_{u,i}^{(0)} = \mathbf{W} \odot \vec{x}_{u,i}$, where $\mathbf{W}$ is the same parameter as in Eq. (3). $\mathbf{W}_a \in \mathbb{R}^{d \times d'}$ is the corresponding linear transformation's weight matrix and σ is the sigmoid activation function. Note that **Aggregate**($\cdot$) can be a function such as *Mean*, *Sum*, *Max*, etc. We use the simple mean function here and leave other functions for future exploration.

To better identify whether the selected core node $v_{u,i}$ is the center of the cluster, we introduce a cluster-aware attention $\alpha_{i,j}$. However, the cluster information of $v_{u,i}$ is not explicitly provided. To this end, we intuitively define the receptive field of cluster c as $v_{u,i}$'s k -hop neighborhoods, with the assumption that each node $v_{u,i}$ in the graph is likely to be a core node forming a cluster c . Then, we calculate a cluster-aware attention score using the embedding of each node in the assumed cluster and the cluster embedding $\vec{x}_{u,i}^{(c)}$ (i.e., the average value of all nodes' embeddings in the cluster) as:

$$\alpha_{i,j} = \text{Attention}_c \left(\mathbf{W}_c \odot \vec{x}_{u,i} \| \mathbf{W}_c \odot \vec{x}_{u,i}^{(c)} \| \mathbf{W}_c \odot \vec{x}_{u,j}; j \in \mathcal{N}_{u,i}^{(c)} \right), \quad (7)$$

where $\text{Attention}_c(\cdot)$ is implemented by a feed-forward neural network with a sigmoid activation function. $\|$ is the concatenation operator. $\mathcal{N}_{u,i}^{(c)}$ is the node set in the assumed cluster c , and $\mathbf{W}_c \in \mathbb{R}^d$

is a learnable transformation vector. Particularly, if G_u contains several sub-graphs, the GCN operation should be separately conducted over each sub-graph to update node representations.

After obtaining cluster-aware code embeddings $\{\vec{x}'_{u,i}\}_{i=1}^n$, we divide them into several true clusters. Numerous clustering methods have already been proposed [45]. In our implementation, a popular K-means algorithm [11] is performed over $\{\vec{x}'_{u,1}, \vec{x}'_{u,2}, \dots, \vec{x}'_{u,n}\}$, and generates C_k clusters. The corresponding clustering index list is $\{idx_{u,1}, idx_{u,2}, \dots, idx_{u,n}\}$. In particular, the earliest submission in each cluster is considered the core signal and the remaining codes in the same cluster are identified as weak signals.

3.2 Adaptive PKT Stage

After performing the identification process, we can obtain the identifications of unwanted codes corresponding to unrelated submissions and the cluster indexes for each core signal from the significant submission and the weak signals from minor modifications. Leveraging these noise indicators, this stage aims to improve the backbone PKT model under the noisy scenario as much as possible.

To facilitate the subsequent calculation, we transform each type of submission signal into the corresponding prompt $\vec{p}_u \in \mathbb{R}^{2d'}$. Concretely, for the core submission with the code representation $\vec{x}'_{u,i}$ and each weak submission with the code representation $\vec{x}'_{u,j}$ in the same cluster c , let $[\mathbf{W}_p \odot \vec{x}'_{u,i}^{(c)} \|\vec{0}^{d'}]$ and $[\vec{0}^{d'} \|\mathbf{W}_p \odot \vec{x}'_{u,j}^{(c)}]$ be their prompts, respectively, where $\vec{0}^{d'}$ denotes a d' -dimensional zero vector. For each unrelated submission with embedding $\vec{x}'_{u,r}$, its prompt is defined as $[\mathbf{W}_p \odot \vec{x}'_{u,r}^{(c)} \|\mathbf{W}_p \odot \vec{x}'_{u,r}^{(c)}]$. $\mathbf{W}_p \in \mathbb{R}^{d'}$ is a learnable transform vector.

3.2.1 Tuning Adaptor for PKT. Based on the prompt indicators, we employ a light and efficient tuning strategy to combine Coda with a refined original PKT model whose parameters are frozen, with the consideration of efficiency.

Inspired by LoRA [14], we implement a tuning adaptor using two trainable rank decomposition matrices, $\mathbf{W}_A \in \mathbb{R}^{b \times 2d'}$ and $\mathbf{W}_B \in \mathbb{R}^{b \times 2d'}$, which are integrated with the CE and KE modules in the backbone PKT model, as shown in Figure 3 (f). The backbone PKT model's parameters are kept frozen. Specifically, the adaptor receives the identification prompt $\vec{p}_{u,t}$ at each step t to sense different types of coding signals (including noisy signals and core submissions) and generates correction signals tailored to various contexts (i.e., the knowledge state and submission at step t). The correction signal is added to the original knowledge state $\vec{h}_{u,t}$ output from the refined KE component of the backbone PKT model to generate the corrected knowledge state as follows:

$$\vec{h}'_{u,t} = \vec{h}_{u,t} + \vec{h}'_{u,t,a}, \quad \vec{h}'_{t,u,a} = (\mathbf{W}_A^T \cdot \mathbf{W}_B) \odot \vec{p}_{u,t}, \quad (8)$$

where $\vec{h}'_{u,t,a}$ is the correction signal. It adjusts the noisy state $\vec{h}_{u,t}$ at each dimension and generates a denoised knowledge state $\vec{h}'_{u,t}$.

To optimize the adaptor effectively, we summarize the characteristics of unwanted and weak signals as two mathematical constraints to control the knowledge state correction, as shown in Figure 3 (g). The first consideration is that unwanted signals are unrelated to programming practice and cannot reflect learners' actual knowledge states. To correct the biased knowledge states

from these unrelated signals in the refined PKT model, we design a constraint that requires the knowledge state updated with irrelevant signal data to be similar to the knowledge state after the most recent question-related attempt (denoted as t^+). Related attempts include any core/weak submissions. This constraint is modeled with the following loss function:

$$\mathcal{L}_{u,t}^{uw} = D_{\text{KL}}(\vec{h}'_{u,t}, \vec{h}'_{u,t^+}), \quad (9)$$

where $D_{\text{KL}}(\cdot)$ denotes Kullback Leibler (KL) divergence [1] to close the gap between two knowledge states. Note that $\mathcal{L}_{u,t}$ is omitted if there are no question-related attempts before the attempt step t .

For each weak submission t , the learner's knowledge state should not undergo significant changes and should be close to the state corresponding to a core submission. This can be formalized as:

$$\mathcal{L}_{u,t}^{\text{weak}} = D_{\text{KL}}(\vec{h}'_{u,t}, \vec{h}'_{u,\text{core}}), \quad (10)$$

where $\vec{h}'_{u,\text{core}}$ is the corrected knowledge state at core submission corresponding to the weak submission at the same cluster.

In summary, the tuning adaptor requires the following noise feature-based loss function:

$$\mathcal{L}_{u,t}^{\text{adaptor}} = \begin{cases} \mathcal{L}_{u,t}^{uw}, & \text{submission } x_{u,t} \text{ is unrelated,} \\ \mathcal{L}_{u,t}^{\text{weak}}, & \text{submission } x_{u,t} \text{ is weak.} \end{cases} \quad (11)$$

3.2.2 Navigational Regularization. To ensure that the adaptor accurately guides the backbone PKT model without adverse effects, we introduce an additional navigational regularization term to constrain the optimization. Specifically, using a Stochastic Gradient Descent (SGD) optimization strategy [33], the model parameters are updated at each training batch. We propose that the model prediction accuracy after tuning at training batch b should not be worse than the last batch ($b-1$). Let θ_b denote the model parameters after the b -th batch; this regularization term can be formalized as:

$$\mathcal{L}_b^{\text{nav}} = \left| \sum_{u=1}^N \sum_{t=1}^{T_u} \mathcal{L}_{u,t}^{\text{pkt}}(\theta_b) - \sum_{u=1}^N \sum_{t=1}^{T_u} \mathcal{L}_{u,t}^{\text{pkt}}(\theta_{b-1}) \right|, \quad (12)$$

where $\mathcal{L}_{u,t}^{\text{pkt}}(\theta_b)$ denote the prediction loss on one training data $(q_{u,t}, c_{u,t}, x_{u,t}, r_{u,t})$ based on the updated model parameter θ_b . $|\cdot|$ is the absolute value operator. The Eq. (12) is intuitive, but computationally complex, and it requires a prediction based on all the training data at each batch. Therefore, we give the upper bound of Eq. (12) with the consideration of the computational complexity and storage space in actual implementation based on **Property 1**:

Property 1. $\mathcal{L}_b^{\text{nav}}$ is bounded as follows:

$$\mathcal{L}_b^{\text{nav}} \leq \left| \left(\nabla_{\theta_{b-1}} \mathcal{L}^{\text{pkt}}(\theta_{b-1}) \right)^T \right| \cdot |\delta\theta_{b-1}| + |\delta\theta_{b-1}|^T \cdot |J^T J| \cdot |\delta\theta_{b-1}|, \quad (13)$$

where $\nabla_{\theta_{b-1}} \mathcal{L}^{\text{pkt}}(\theta_{b-1})$ is the training gradient after the $(b-1)$ -th batch tuning, $\delta\theta_{b-1}$ denote the parameter change between the b -th batch and $(b-1)$ -th batch, i.e., $\delta\theta_{b-1} = \theta_b - \theta_{b-1}$. J is the Jacobian matrix [41]. In implement, the Jacobian matrix J is denoted using the training gradient $\nabla_{\theta_{b-1}} \mathcal{L}^{\text{pkt}}(\theta_{b-1})$. Compared to Eq. (12), the Eq. (13) is calculated once per training batch, requiring additional storage space for storing model parameters and gradients, which is acceptable in practice. See the Appendix A for the full proof.

3.3 Optimization

Overall, at each training batch b with programming records of a batch of learners $\mathcal{U}_b \sim \mathcal{U}$, putting the prediction objective (Eq. (2)), the tuning objective (Eq. (11)) and the navigation regularization (Eq. (13)) together, we have the following training loss function:

$$\mathcal{L}^{Coda} = \sum_{\mathcal{U}_b \sim \mathcal{U}} \left(\sum_{u \in \mathcal{U}_b} \sum_{t=1}^{T_u} \left(\mathcal{L}_{u,t}^{pkt} + \mathcal{L}_{u,t}^{adaptor} \right) + \mathcal{L}_b^{nav} \right). \quad (14)$$

3.4 Computational Complexity

During the training phase, the code graph is constructed only once for each learner u 's sequence l_u . The primary computational complexity arises from calculating the adjacency matrix (see Eq. (3) and Eq. (4)), which has a complexity of $O(T_u \times T_u)$, where T_u is the length of the programming sequence. When identifying irrelevant noise, we introduce an auxiliary task with a complexity of $O(w)$, where w represents the average number of correct submissions per question. Identifying weak submissions mainly involves the GCN, with a complexity of approximately $O(T_u \times L)$, where L is the number of layers in the GCN, and the clustering complexity is $O(T_u \times C_k)$, where C_k is the number of clusters. When tuning the backbone PKT model, the primary complexity arises from predicting the learner's performance for each submission step, with a complexity of $O(T_u)$. Therefore, the overall complexity of modeling each learner during the entire training phase is $O(T_u \times T_u) + O(w) + O(T_u \times L) + O(T_u \times C_k) + O(T_u)$. During the test phase, for a learner u , the code graph needs to be constructed at each step, resulting in a complexity of $O(\sum_{i=1}^{T_u} i^2) + O(T_u \times w) + O(\sum_{i=1}^{T_u} i \times L) + O(\sum_{i=1}^{T_u} i \times C_k) + O(\sum_{i=1}^{T_u} i)$.

4 Experiments

We conduct comprehensive experiments to address the following research questions (RQs): **RQ1** Can Coda enhance the accuracy of knowledge state estimation for backbone PKT models, thereby improving the prediction of student performance? **RQ2** How do the various components designed in Coda influence its performance? **RQ3** What is the effectiveness of code graph modeling? **RQ4** How powerful is Coda in tracing the evolution of knowledge states?

4.1 Datasets

We evaluate the effectiveness of Coda using four real-world public datasets: BePKT¹, AtCoder², BePKT_C++ and AtCoder_C. These datasets contain learner programming records, including submitted code and feedback on judgments, as well as question-knowledge correlations where each question is linked to a specific knowledge concept. Specifically, BePKT and AtCoder include multiple programming languages (e.g., C and C++), while BePKT_C++ and AtCoder_C are subsets of BePKT and AtCoder, respectively, that focus exclusively on C++ and C submissions. The reason why we construct the two subsets is the baseline (i.e., PST [21]) introduced in § 4.2.1 only supports the single language programming scenario. The complete statistics of these datasets are summarized in Table 2. Detailed dataset information and preprocessing steps are provided in Appendix B.

¹<https://drive.google.com/drive/folders/1Jt6f0MV1paGLctJqxHtdF1Vh2mUnsoV?usp=sharing>

²https://github.com/IBM/Project_CodeNet

4.2 Experimental Setups

4.2.1 Baselines. For baselines, we first select some traditional knowledge tracing models that are classic but not specifically tailored for programming scenarios, i.e., DKT [31], AKT [8], and LPKT [34]. Then, we select several well-adopted PKT models, i.e., DKT+ [39], PDKT [54], Code-DKT [36], Help-DKT [23], PST [21] and ECKT [48]. Notably, PST requires running in a single programming language scenario (i.e., the BePKT_C++ and AtCoder_C datasets). The descriptions of baselines are listed as follows:

- **DKT** leverages RNN to assess knowledge mastery by iteratively predicting learner next performance.
- **AKT** is the context-aware attentive KT model. It uses the two self-attentive encoders to learn context-aware representations of the exercises and answers.
- **LPKT** monitors learners' knowledge state through directly modeling their learning process with a basic chain: *exercise—answer time—answer*.
- **DKT+** uses abstract syntax trees (AST) [2, 21] to represent programs and uses recurrent neural networks to track learners' knowledge levels.
- **PDKT** uses a double-sequence model with exponential decay attention to model problem and code sequences.
- **Code-DKT** constructs the AST and adopts both node embedding and path embedding to effectively model the problem text.
- **Help-DKT** introduces an interpretable cognitive model that integrates programming error classification as a conceptual indicator into a personalized feedback-based matrix, aiming to estimate student ability at the conceptual level.
- **PST** builds a code information graph and a code tracing graph containing rich structural and semantic information from code contents. Additionally, it takes into account code modification content between consecutive submissions to capture knowledge state changes.
- **ECKT** integrates code features with questions' knowledge concepts and descriptions generated by an LLM from codes. In this paper, we generate knowledge concept embeddings based on the original datasets and generate each question description embedding by mean-pooling all codes related to the question.

4.2.2 Evaluation Metrics. As knowledge states cannot be directly observed in practice, it is common to indirectly assess PKT tasks through predicting learner response performance on test datasets [7, 20, 21]. To evaluate model performance, we adopt ACC, AUC and RMSE as metrics from the perspectives of classification and regression following the [6, 46].

4.2.3 Implementation Details. We implement our Coda framework with five well-adopted PKT backbone models, i.e., DKT+ [39], PDKT [54], Code-DKT [36], Help-DKT [23] and PST [21]. We call them Coda-X, e.g., Coda-PDKT and Coda-Help-DKT. We split 70%, 10% and 20% programming sequences of each dataset as the training set, the valid set, and the test set, respectively. We utilize a pre-trained Code-Bert³ to encode input codes for each baseline with the setting of dimension d is 768. The setups for training PKT backbone models refer to the original papers and each backbone is trained on training data. We implement our Coda-X models by

³<https://github.com/microsoft/CodeBERT>

Table 1: Performance comparison. The best performance is highlighted in bold, and $\uparrow$ ($\downarrow$) means the higher (lower) score the better (worse) performance, the same as below.

Datasets	Metrics	KT Models			PKT Models					Coda					
		DKT	AKT	LPKT	DKT+	PDKT	Code-DKT	Help-DKT	PST	ECKT	Coda-DKT+	Coda-PDKT	Code-Coda-DKT	Coda-Help-DKT	Coda-PST
BePKT	AUC (%) ↑	52.77	54.12	53.69	53.96	55.32	56.14	60.10	-	59.63	56.34	59.74	59.42	62.31	-
	F1-score (%) ↑	9.30	12.51	10.07	10.13	14.68	21.03	31.65	-	34.32	26.96	31.39	34.22	36.03	-
	RMSE (%) ↓	44.67	30.59	31.78	42.93	40.01	34.56	31.96	-	35.44	41.24	39.03	34.25	31.33	-
AtCoder	AUC (%) ↑	52.37	53.34	54.62	52.46	53.57	57.23	58.32	-	59.32	54.02	59.03	62.96	59.63	-
	F1-score (%) ↑	7.00	15.96	22.74	7.02	16.39	27.42	31.44	-	32.40	16.31	34.69	35.37	33.21	-
	RMSE (%) ↓	45.17	41.88	38.82	44.37	39.66	31.63	39.42	-	37.54	43.20	38.27	30.32	38.36	-
BePKT_C++	AUC (%) ↑	52.96	54.17	53.29	54.17	55.29	55.63	60.58	60.21	60.02	57.89	59.30	58.83	62.97	63.75
	F1-score (%) ↑	8.37	10.24	10.01	10.32	14.29	18.21	33.75	28.57	27.92	27.16	30.65	33.49	37.19	37.40
	RMSE (%) ↓	44.28	43.77	43.82	42.09	40.66	34.08	31.30	31.07	32.03	40.89	39.57	33.65	30.97	30.20
AtCoder_C	AUC (%) ↑	52.51	53.07	53.27	53.12	54.14	56.54	59.83	64.17	64.43	54.64	57.36	62.14	60.62	65.94
	F1 (%) ↑	7.84	10.67	12.42	8.49	12.33	24.99	33.89	51.74	52.04	18.31	32.33	40.67	36.08	61.28
	RMSE (%) ↓	43.20	42.96	41.77	42.20	41.30	38.41	38.03	40.80	40.01	41.11	40.93	38.28	37.90	39.26

Table 2: The statistics of four programming datasets.

Dataset	BePKT	AtCoder	BePKT_C++	AtCoder_C
#learners	479	5,523	447	5,416
#questions	423	832	146	832
#knowledge concepts	83	832	83	832
#submissions	27,679	197,419	4,278	27,297
#submissions per learner	57.78	35.74	9.57	5.04

tuning these refined baselines. The parameter ϵ in Eq. (4) is sampled from the “0.2/0.4/0.5/0.6/0.8 of the square of the sequence length” to regulate the sparsity of the code graph. The GCN layers L for Eq. (6) and cluster-aware hop k in Eq. (7) are both 2, following the related works [3]. The number of clusters of k-means C_k is searched in [1, 3, 5]. The dimension d' is equal to the dimension d of original code embedding and b in Eq. (8) is $d'/2$. Furthermore, we set the mini-batch size as 32 and the learning rate is searched in $[1e^{-3}, 1e^{-2}, 1e^{-1}]$ for each model. For training, all network parameters are initialized with Xavier initialization [9]. Each model is implemented by PyTorch [30] and optimized by Adam optimizer [17]. All experiments are run on a cluster of Linux servers with NVIDIA TITAN V100 GPUs. Code is available at <https://github.com/bigdata-ustc/Coda-PKT>.

4.3 Next Attempt Performance Prediction (RQ1)

To evaluate model effectiveness, we compare all methods based on learners’ subsequent attempt prediction accuracy, as depicted in Table 1. We can see that: (1) Compared to traditional KT models, the PKT models and Coda-Xs show superior performance across all metrics. Traditional KT models are designed for a single submission per problem and ignore code information, leading to biases in scenarios with multiple submissions. (2) In comparison to PKT models, our Coda-Xs demonstrate better prediction accuracy. For example, on the BePKT_C++ dataset, Coda improves Help-DKT and PST by 3.44% and 8.83% in terms of F1 score accuracy, respectively. It reveals that our Coda to denoising programming sequences is both justified and effective. (3) Most models perform better on two single language datasets, suggesting that longer programming sequences have more noise, thus limiting model performance.

4.4 Investigation of Key Components (RQ2)

To validate the effectiveness of each Coda design, we conduct an ablation study on Coda-PST using the BePKT_C++ dataset. Specifically, we implement various Coda variants including “w/o iso” by ignoring the isolated nodes (§ 3.1.2), “w/o sim” by removing the

Table 3: Ablation study on BePKT_C++.

Methods	AUC (%) $\uparrow$	F1 (%) $\uparrow$	RMSE (%) $\downarrow$
Coda-PST	63.75	37.40	30.20
w/o iso	62.57	36.92	30.60
w/o sim	63.32	37.23	30.63
w/o uw	62.21	37.02	31.43
w/o gcn	63.00	36.92	30.74
w/o weak	61.58	34.75	32.30
w/o nav	63.60	37.34	30.34

similarity calculation between codes and solutions (i.e., Eq. (5)), “w/o uw” by discarding the identification of unwanted signals (i.e., Eq. (9)), “w/o gcn” by skipping the clustering-aware GCN (i.e., Eq. (6)), “w/o weak” by discarding the identification of weak signals (i.e., Eq. (10)), and “w/o nav” by discarding the navigational regularization (i.e., Eq. (13)). Table 3 lists the prediction performance. From the table, we can see that each component of Coda significantly enhances the predictive performance of the PKT task.

4.5 Analysis of the Code Graph (RQ3)

The parameter ϵ in Eq.(4) determines both the sparsity of the constructed code graph and the subsequent prediction accuracy. To explore the impact of varying graph sparsity, we train the Coda-Help-DKT on several sparse code graphs where ϵ is set to “ p of the square of the sequence length”, with $p \in \{0.2, 0.4, 0.5, 0.6, 0.8\}$. Figure 5 illustrates the prediction scores for different sparse setups on the BePKT_C++ dataset, clearly demonstrating that the model’s performance improves with increasing ϵ when ϵ is small. However, once ϵ exceeds a certain threshold (e.g., 0.5 of the square of the sequence length for Coda-PDKT), the performance gains become less pronounced.

4.6 Visualization of Knowledge Tracing (RQ4)

One of the most practical aspects of PKT is its ability to diagnose proficiency of specific knowledge concepts for learners at each attempt during programming. To gain deeper insights into how Coda tracks the evolution of programming knowledge states, this section illustrates the tracing process using various models across a learning sequence. Specifically, for a trained PKT model, we extract the knowledge proficiency on question-related concept c from the c -th dimensional element in the knowledge state $\vec{h}_{u,t}$ inspired by [26, 47]. We randomly sample a real programming sub-sequence of a learner ($id = 320$) response to the question ($id = 1304$) that tests

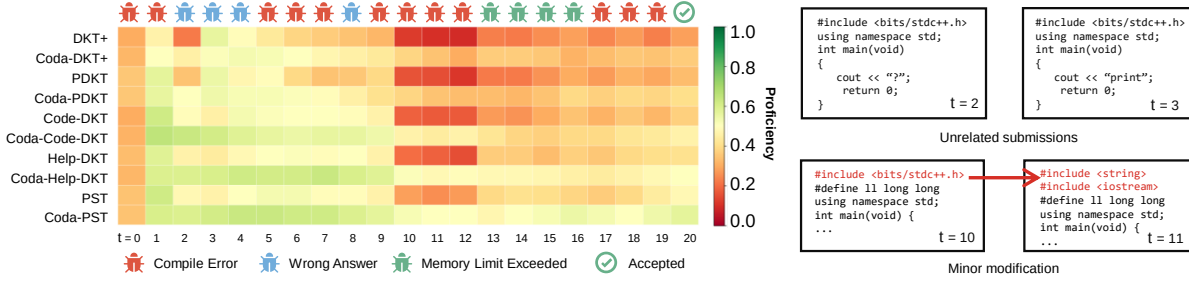


Figure 4: Visualization of Code-X and original PKT models tracing knowledge of a learner along the practice sequence.

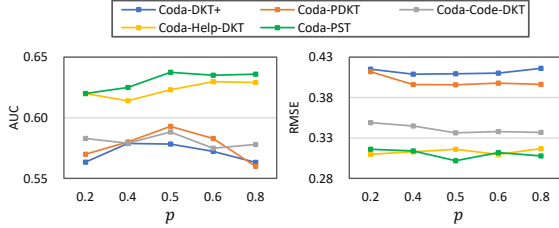


Figure 5: Performance under different sparse levels of the code graph on BePKT_C++.

“Addition of Large Numbers” from BePKT_C++, and visualize the evolution of proficiency on the concept tested by the question. Figure 4 shows the tracing process learned by several models.

From the visualization, it is obvious that all Coda-X models demonstrate more stable knowledge tracing after the initial state ($t = 0$) compared to original PKT models. The baselines may be susceptible to noise stemming from noisy submissions. Upon observing the learner’s submitted codes (presented in the right part of Figure 4), it is evident that baselines are misled by noisy signals. For instance, at $t = 2$ and $t = 3$, the learner’s codes are unrelated to the question. Coda-Xs effectively identifies noise submissions, thus preventing an underestimation of student knowledge due to these buggy examples. In contrast, PKT baselines are misled by irrelevant data. Similarly, at $t = 10$ and $t = 11$, the learner makes minor code modifications, indicating that his/her knowledge state should be relatively stable. While these submissions have no explicit bugs, they are redundant to PKT models. Coda-Xs remain unaffected by these minor changes due to its ability to identify and correct noisy signals, while the original PKT models continue to be misled, consistently underestimating learner knowledge proficiency.

5 Related Work

In the field of educational data mining, Programming Knowledge Tracking (PKT) has taken on a significant role, focusing on the assessment. PKT aims to dynamically diagnose learners’ proficiency on specific programming knowledge. It extends traditional knowledge tracking (KT) techniques [8, 28, 31, 34, 37, 50] and provides important technology for diagnosing students’ coding abilities by integrating code features related to open programming environments. Existing PKT methods mainly consist of three key modules: code encoder (CE), knowledge estimation (KE), and predictor. For example, DKT+ [39] first uses Code2Vec technology to encode the code submitted by students, then models the student’s answer sequence using LSTM [13], and uses the final representation to

model the student’s programming ability status. PDKT [54] incorporates representations of questions and knowledge points on this basis to enhance the capabilities of the knowledge modeler. Code-DKT [36] utilizes attention mechanisms to automatically extract and select domain-specific code features to extend DKT. HELP-DKT [23] combines students’ original code encoding with error category information to enhance the capabilities of the knowledge modeler. PST [21] models differences between students’ submitted codes in order to model their programming skills and knowledge, achieving state-of-the-art performance. ECKT [48] integrates code features with questions’ knowledge concepts and descriptions generated by a large language model (LLM) from the code. GPKT [42] learns informative code and question representations from a code-question-knowledge network for better prediction. However, all these methods are limited by noise in students’ answer sequences, which affects their applicability in real-world environments.

6 Conclusion and Future Research

In this study, we aim to address the prevalent challenges in programming knowledge tracing (PKT), with a particular focus on the impact of noisy signals encountered during long-term programming activities. We identify two prominent types of noisy signals: unwanted signals from unrelated submissions and weak signals from minor modifications. Both types significantly hinder the accurate representation of a learner’s knowledge state. To mitigate the impact of noise on PKT models, we propose the Coda framework. This framework enhances existing PKT solutions by enabling them to identify noisy signals and adaptively reduce their negative effects. Our approach involves transforming code sequence modeling issues into code graph modeling problems by leveraging the semantic relationships in the code. Additionally, we introduce a lightweight tuning adaptor to seamlessly integrate with the PKT task. Extensive experiments on four real-world datasets validate the effectiveness of our Coda model. Future research will identify additional types of noise and incorporate specific buggy feedback to achieve more comprehensive denoising for the programming knowledge tracing task.

Acknowledgments

This research was partially supported by grants from the National Natural Science Foundation of China (Grants No. 62337001), the Key Technologies R & D Program of Anhui Province (No. 202423k09020039), and the Fundamental Research Funds for the Central Universities.

References

- [1] Alexander A Alemi, Ian Fischer, Joshua V Dillon, and Kevin Murphy. 2016. Deep variational information bottleneck. *arXiv preprint arXiv:1612.00410* (2016).
- [2] Uri Alon, Meital Zilberstein, Omer Levy, and Eran Yahav. 2018. code2vec: Learning Distributed Representations of Code. *arXiv:1803.09473* [cs.LG]
- [3] Jianxin Chang, Chen Gao, Yu Zheng, Yiqun Hui, Yanan Niu, Yang Song, Depeng Jin, and Yong Li. 2021. Sequential recommendation with graph neural networks. In *Proceedings of the 44th international ACM SIGIR conference on research and development in information retrieval*. 378–387.
- [4] Yu Chen, Lingfei Wu, and Mohammed Zaki. 2020. Iterative deep graph learning for graph neural networks: Better and robust node embeddings. *Advances in neural information processing systems* 33 (2020), 19314–19326.
- [5] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, and Ming Zhou. 2020. CodeBERT: A Pre-Trained Model for Programming and Natural Languages. *arXiv:2002.08155* [cs.CL]
- [6] Weibo Gao, Qi Liu, Zhenya Huang, Yu Yin, Haoyang Bi, Mu-Chun Wang, Jianhui Ma, Shijin Wang, and Yu Su. 2021. Red: Relation map driven cognitive diagnosis for intelligent education systems. In *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 501–510.
- [7] Weibo Gao, Qi Liu, Hao Wang, Linan Yue, Haoyang Bi, Yin Gu, Fangzhou Yao, Zheng Zhang, Xin Li, and Yuanjing He. 2024. Zero-1-to-3: Domain-Level Zero-Shot Cognitive Diagnosis via One Batch of Early-Bird Students towards Three Diagnostic Objectives. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 8417–8426.
- [8] Aritra Ghosh, Neil Heffernan, and Andrew S Lan. 2020. Context-aware attentive knowledge tracing. In *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*. 2330–2339.
- [9] Xavier Glorot and Yoshua Bengio. 2010. Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the thirteenth international conference on artificial intelligence and statistics*. JMLR Workshop and Conference Proceedings, 249–256.
- [10] Siqi Han, Yu Wang, and Xuesong Lu. 2023. ErrorCLR: Semantic Error Classification, Localization and Repair for Introductory Programming Assignments. In *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 1345–1354.
- [11] John A Hartigan and Manchek A Wong. 1979. Algorithm AS 136: A k-means clustering algorithm. *Journal of the royal statistical society. series c (applied statistics)* 28, 1 (1979), 100–108.
- [12] Jonathan Ho, Ajay Jain, and Pieter Abbeel. 2020. Denoising diffusion probabilistic models. *Advances in neural information processing systems* 33 (2020), 6840–6851.
- [13] Sepp Hochreiter and Jürgen Schmidhuber. 1997. Long short-term memory. *Neural computation* 9, 8 (1997), 1735–1780.
- [14] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2021. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685* (2021).
- [15] Yilun Jin, Kai Chen, and Qiang Yang. 2023. Transferable graph structure learning for graph-based traffic forecasting across cities. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 1032–1043.
- [16] Jin-Hwa Kim, Kyoung-Woon On, Woosang Lim, Jeonghee Kim, Jung-Woo Ha, and Byoung-Tak Zhang. 2016. Hadamard product for low-rank bilinear pooling. *arXiv preprint arXiv:1610.04325* (2016).
- [17] Diederik P Kingma and Jimmy Ba. 2014. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980* (2014).
- [18] Philip IS Lei and António José Mendes. 2021. A systematic literature review on knowledge tracing in learning programming. In *2021 IEEE Frontiers in Education Conference (FIE)*. IEEE, 1–7.
- [19] Haoyang Li, Xin Wang, Ziwei Zhang, Jianxin Ma, Peng Cui, and Wenwu Zhu. 2021. Intention-aware sequential recommendation with structured intent transition. *IEEE Transactions on Knowledge and Data Engineering* 34, 11 (2021), 5403–5414.
- [20] Mingjia Li, Hong Qian, Jinglan Lv, Mengliang He, Wei Zhang, and Aimin Zhou. 2025. Foundation model enhanced derivative-free cognitive diagnosis. *Frontiers of Computer Science* 19, 1 (2025), 191318.
- [21] Ruixin Li, Yu Yin, Le Dai, Shuanghong Shen, Xin Lin, Yu Su, and Enhong Chen. 2022. PST: Measuring Skill Proficiency in Programming Exercise Process via Programming Skill Tracing. In *Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 2601–2606.
- [22] Rongfan Li, Ting Zhong, Xinke Jiang, Goce Trajcevski, Jin Wu, and Fan Zhou. 2022. Mining spatio-temporal relations via self-paced graph contrastive learning. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 936–944.
- [23] Yu Liang, Tianhao Peng, Yanjun Pu, and Wenjun Wu. 2022. HELP-DKT: an interpretable cognitive model of how students learn programming based on deep knowledge tracing. *Scientific Reports* 12, 1 (2022), 1–11.
- [24] Jiayu Liu, Zhenya Huang, Zhiyuan Ma, Qi Liu, Enhong Chen, Tianhuang Su, and Haifeng Liu. 2023. Guiding Mathematical Reasoning via Mastering Commonsense Formula Knowledge. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 1477–1488.
- [25] Naïming Liu, Zichao Wang, Richard G Baraniuk, and Andrew Lan. 2022. GPT-based Open-Ended Knowledge Tracing. *arXiv preprint arXiv:2203.03716* (2022).
- [26] Qi Liu, Zhenya Huang, Yu Yin, Enhong Chen, Hui Xiong, Yu Su, and Guoping Hu. 2019. Ekt: Exercise-aware knowledge tracing for student performance prediction. *IEEE Transactions on Knowledge and Data Engineering* 33, 1 (2019), 100–115.
- [27] Shuo Liu, Junhao Shen, Hong Qian, and Aimin Zhou. 2024. Inductive Cognitive Diagnosis for Fast Student Learning in Web-Based Intelligent Education Systems. In *Proceedings of the ACM on Web Conference 2024*. Singapore, 4260–4271.
- [28] Haiping Ma, Yong Yang, Chuan Qin, Xiaoshan Yu, Shangshang Yang, Xingyi Zhang, and Hengshu Zhu. 2024. HD-KT: Advancing Robust Knowledge Tracing via Anomalous Learning Interaction Detection. In *Proceedings of the ACM on Web Conference 2024*. 4479–4488.
- [29] Andy Nguyen, Christopher Piech, Jonathan Huang, and Leonidas Guibas. 2014. Codewebs: scalable homework search for massive open online programming courses. In *Proceedings of the 23rd international conference on World wide web*. 491–502.
- [30] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. 2019. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems* 32 (2019).
- [31] Chris Piech, Jonathan Bassen, Jonathan Huang, Surya Ganguli, Mehran Sahami, Leonidas J Guibas, and Jascha Sohl-Dickstein. 2015. Deep knowledge tracing. *Advances in neural information processing systems* 28 (2015).
- [32] Ruchir Puri, David S. Kung, Geert Janssen, Wei Zhang, Giacomo Domeniconi, Vladimir Zolotov, Julian Dolby, Jie Chen, Mihir Choudhury, Lindsey Decker, Veronika Thost, Luca Buratti, Saurabh Pujar, Shyam Ramji, Ulrich Finkler, Susan Malaika, and Frederick Reiss. 2021. CodeNet: A Large-Scale AI for Code Dataset for Learning a Diversity of Coding Tasks. *arXiv:2105.12655* [cs.SE]
- [33] Sebastian Ruder. 2016. An overview of gradient descent optimization algorithms. *arXiv preprint arXiv:1609.04747* (2016).
- [34] Shuanghong Shen, Qi Liu, Enhong Chen, Zhenya Huang, Wei Huang, Yu Yin, Yu Su, and Shijin Wang. 2021. Learning Process-consistent Knowledge Tracing. In *KDD '21: The 27th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, Virtual Event, Singapore, August 14–18, 2021*, Feida Zhu, Beng Chin Ooi, and Chunyan Miao (Eds.). ACM, 1452–1460. doi:10.1145/3447548.3467237
- [35] Shuanghong Shen, Qi Liu, Zhenya Huang, Yonghe Zheng, Minghao Yin, Minjuan Wang, and Enhong Chen. 2024. A survey of knowledge tracing: Models, variants, and applications. *IEEE Transactions on Learning Technologies* (2024).
- [36] Yang Shi, Min Chi, Tiffany Barnes, and Thomas Price. 2022. Code-DKT: A Code-based Knowledge Tracing Model for Programming Tasks. *arXiv preprint arXiv:2206.03545* (2022).
- [37] Fei Wang, Weibo Gao, Qi Liu, Jiatong Li, Guanhao Zhao, Zheng Zhang, Zhenya Huang, Mengxiao Zhu, Shijin Wang, Wei Tong, et al. 2024. A survey of models for cognitive diagnosis: New developments and future directions. *arXiv preprint arXiv:2407.05458* (2024).
- [38] Fei Wang, Qi Liu, Enhong Chen, Zhenya Huang, Yu Yin, Shijin Wang, and Yu Su. 2022. NeuralCD: a general framework for cognitive diagnosis. *IEEE Transactions on Knowledge and Data Engineering* 35, 8 (2022), 8312–8327.
- [39] Lisa Wang, Angela Sy, Larry Liu, and Chris Piech. 2017. Deep knowledge tracing on programming exercises. In *Proceedings of the fourth (2017) ACM conference on learning@ scale*. 201–204.
- [40] Xiao Wang, Meiqi Zhu, Deyu Bo, Peng Cui, Chuan Shi, and Jian Pei. 2020. Amgn: Adaptive multi-channel graph convolutional networks. In *Proceedings of the 26th ACM SIGKDD International conference on knowledge discovery & data mining*. 1243–1253.
- [41] Bogdan M Wilamowski and Hao Yu. 2010. Improved computation for Levenberg–Marquardt training. *IEEE transactions on neural networks* 21, 6 (2010), 930–937.
- [42] Yaqiang Wu, Hui Zhu, Chenyang Wang, Fujian Song, Haiping Zhu, Yan Chen, Qinghua Zheng, and Feng Tian. 2024. Programming Knowledge Tracing Based on Heterogeneous Graph Representation. *Knowledge-Based Systems* (2024), 112161.
- [43] Zhengyang Wu, Ming Li, Yong Tang, and Qingyu Liang. 2020. Exercise recommendation based on knowledge concept prediction. *Knowledge-Based Systems* 210 (2020), 106481.
- [44] Feng Xia, Ke Sun, Shuo Yu, Abdul Aziz, Liangtian Wan, Shirui Pan, and Huan Liu. 2021. Graph learning: A survey. *IEEE Transactions on Artificial Intelligence* 2, 2 (2021), 109–127.
- [45] Rui Xu and Donald Wunsch. 2005. Survey of clustering algorithms. *IEEE Transactions on neural networks* 16, 3 (2005), 645–678.
- [46] Fangzhou Yao, Qi Liu, Linan Yue, Weibo Gao, Jiatong Li, Xin Li, and Yuanjing He. 2024. Adard: An adaptive response denoising framework for robust learner modeling. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 3886–3895.
- [47] Yu Yin, Le Dai, Zhenya Huang, Shuanghong Shen, Fei Wang, Qi Liu, Enhong Chen, and Xin Li. 2023. Tracing Knowledge Instead of Patterns: Stable Knowledge Tracing with Diagnostic Transformer. In *Proceedings of the ACM Web Conference 2023*. 855–864.

- [48] Yang Yu, Yingbo Zhou, Yaokang Zhu, Yutong Ye, Liangyu Chen, and Mingsong Chen. 2024. ECKT: Enhancing Code Knowledge Tracing via Large Language Models. In *Proceedings of the Annual Meeting of the Cognitive Science Society*, Vol. 46.
- [49] Linan Yue, Qi Liu, Ye Liu, Weibo Gao, Fangzhou Yao, and Wenfeng Li. 2024. Cooperative classification and rationalization for graph generalization. In *Proceedings of the ACM on Web Conference 2024*. 344–352.
- [50] Zheng Zhang, Le Wu, Qi Liu, Jiayu Liu, Zhenya Huang, Yu Yin, Yan Zhuang, Weibo Gao, and Enhong Chen. 2024. Understanding and improving fairness in cognitive diagnosis. *Science China Information Sciences* 67, 5 (2024), 152106.
- [51] Chuang Zhao, Hongke Zhao, Xiaomeng Li, Ming He, Jiahui Wang, and Jianping Fan. 2023. Cross-domain recommendation via progressive structural alignment. *IEEE Transactions on Knowledge and Data Engineering* (2023).
- [52] Hao Zhao, Ming Lu, Anbang Yao, Yiwen Guo, Yurong Chen, and Li Zhang. 2017. Physics inspired optimization on semantic transfer features: An alternative method for room layout estimation. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 10–18.
- [53] Man Zhou, Jie Xiao, Yifan Chang, Xueyang Fu, Aiping Liu, Jinshan Pan, and Zheng-Jun Zha. 2021. Image de-raining via continual learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 4907–4916.
- [54] Renny Zhu, Dongxiang Zhang, Chengcheng Han, Ming Gaol, Xuesong Lu, Weining Qian, and Aoying Zhou. 2022. Programming knowledge tracing: A comprehensive dataset and a new model. In *2022 IEEE International Conference on Data Mining Workshops (ICDMW)*. IEEE, 298–307.
- [55] IS Zinovieva, VO Artemchuk, Anna V Iatsyshyn, OO Popov, VO Kovach, Andrii V Iatsyshyn, YO Romanenko, and OV Radchenko. 2021. The use of online coding platforms as additional distance tools in programming education. In *Journal of physics: Conference series*, Vol. 1840. IOP Publishing, 012029.

A Proof

Property 1. $\mathcal{L}_b^{nav}$ is bounded as follows:

$$\mathcal{L}_b^{nav} \leq \left| \left(\nabla_{\theta_{b-1}} \mathcal{L}^{pkt}(\theta_{b-1}) \right)^T \right| \cdot |\delta\theta_{b-1}| + |\delta\theta_{b-1}|^T \cdot |J^T J| \cdot |\delta\theta_{b-1}|, \quad (15)$$

where J is the Jacobian matrix [41]. The proof is as follows:

PROOF. Given the following equations,

$$\mathcal{L}_b^{nav} = \left| \sum_{u=1}^N \sum_{t=1}^{T_u} \mathcal{L}_{u,t}^{pkt}(\theta_b) - \sum_{u=1}^N \sum_{t=1}^{T_u} \mathcal{L}_{u,t}^{pkt}(\theta_{b-1}) \right|, \quad \delta\theta_{b-1} = \theta_b - \theta_{b-1}, \quad (16)$$

and let $\mathcal{L}^{pkt}(\theta_b)$ and $\mathcal{L}^{pkt}(\theta_{b-1})$ denote $\sum_{u=1}^N \sum_{t=1}^{T_u} \mathcal{L}_{u,t}^{pkt}(\theta_b)$ and $\sum_{u=1}^N \sum_{t=1}^{T_u} \mathcal{L}_{u,t}^{pkt}(\theta_{b-1})$ respectively, we take the Taylor expansion of $\mathcal{L}^{pkt}(\theta_{b-1})$ at point θ_{b-1} , which is an infinite sum of terms that are expressed in the form of loss function's derivatives at a single point:

$$\begin{aligned} \mathcal{L}^{pkt}(\theta_{b-1} + \delta\theta_{b-1}) &= \mathcal{L}^{pkt}(\theta_{b-1}) + \left(\nabla_{\theta_{b-1}} \mathcal{L}^{pkt}(\theta_{b-1}) \right)^T \cdot \delta\theta_{b-1} \\ &\quad + \frac{1}{2} (\delta\theta_{b-1})^T \cdot H \cdot \delta\theta_{b-1} + O(\|\delta\theta_{b-1}\|^3), \end{aligned} \quad (17)$$

where H denotes Hessian matrix:

$$H = \nabla_{\theta_{b-1}} \mathcal{L}^{pkt}(\theta_{b-1}). \quad (18)$$

Then $\mathcal{L}_b^{nav}$ can be derived as:

$$\begin{aligned} \mathcal{L}_b^{nav} &= \left| \mathcal{L}^{pkt}(\theta_b) - \mathcal{L}^{pkt}(\theta_{b-1}) \right| \\ &= \left| \mathcal{L}^{pkt}(\theta_{b-1} + \delta\theta_{b-1}) - \mathcal{L}^{pkt}(\theta_{b-1}) \right| \\ &\approx \left| \left(\nabla_{\theta_{b-1}} \mathcal{L}^{pkt}(\theta_{b-1}) \right)^T \cdot \delta\theta_{b-1} + \frac{1}{2} (\delta\theta_{b-1})^T \cdot H \cdot \delta\theta_{b-1} \right|. \end{aligned} \quad (19)$$

It is evident that Eq.(19) significantly reduces the computational demands on each training datum compared to Eq.(17), thereby accelerating the calculation of the total loss. However, the storage requirements associated with Hessian matrices remain substantial [53]. Therefore, drawing inspiration from the Gauss-Newton method [41], we approximate H as $2 \cdot J^T J$ for the Hessian matrix, where J represents the Jacobian matrix. Consequently, we arrive at the final formulation of $\mathcal{L}_b^{nav}$ as follows:

$$\begin{aligned} \mathcal{L}_b^{nav} &\approx \left| \left(\nabla_{\theta_{b-1}} \mathcal{L}^{pkt}(\theta_{b-1}) \right)^T \cdot \delta\theta_{b-1} + (\delta\theta_{b-1})^T \cdot J^T J \cdot \delta\theta_{b-1} \right| \\ &\leq \left| \left(\nabla_{\theta_{b-1}} \mathcal{L}^{pkt}(\theta_{b-1}) \right)^T \right| \cdot |\delta\theta_{b-1}| + |\delta\theta_{b-1}|^T \cdot |J^T J| \cdot |\delta\theta_{b-1}| \end{aligned} \quad (20)$$

where $|\cdot|$ in the Eq. (20) denotes element-wise absolute value in our implementation. The proof is completed. $\square$

Algorithm 1 The training and test of PKT backbone models

Require: Training dataset D_{train} , test dataset D_{test} , learning rate α , number of epochs N

Ensure: Trained PKT backbone model M

- 1: Initialize model parameters W randomly
- 2: **Training started**
- 3: **for** $epoch = 1$ **to** N **do**
- 4: **for all** $l_u \in D_{train}$ **do**
- 5: // l_u denotes a programming sequence of learner u and its length is T_u
- 6: **for** $t = 1$ **to** $T_u - 1$ **do**
- 7: Compute programming performance at the next step $\hat{r}_{u,t+1} = M(l_{u,1:t}; W)$
- 8: // $l_{u,1:t}$ is programming records of u from step 1 to t
- 9: Compute cross-entropy loss $\mathcal{L}_{u,t}^{pkt} = BCE(r_{u,t+1}, \hat{r}_{u,t+1})$
- 10: Update parameters $W = W - \alpha \nabla \mathcal{L}_{u,t}^{pkt}$
- 11: **end for**
- 12: **end for**
- 13: **end for**
- 14: **Training completed**
- 15:
- 16: **Testing started**
- 17: **for all** $l_u \in D_{test}$ **do**
- 18: **for** $t = 1$ **to** $T_u - 1$ **do**
- 19: Compute programming performance at the next step $\hat{r}_{u,t+1} = M(l_u; W)$
- 20: Compute performance metric using $r_{u,t+1}$ and $\hat{r}_{u,t+1}$
- 21: **end for**
- 22: **end for**
- 23: **Testing completed**

B Datasets and Preprocessing

BePKT[54] comprises extensive programming submission records in several languages authored by college students. We retain submission records written in *C*, *C++*, *Python*, and *Java*. **AtCoder** is collected from the online programming competition website *AtCoder.org*, consisting of codes in multiple languages, as published by

Algorithm 3 The Algorithm of Coda (Part 2)

```

1: BEGIN MAIN FUNCTION:
2: Initialize: learning rate  $lr$ ,  $epoch \leftarrow 0$ ,  $BatchSize$ ,  $TotalEpoch$ ,
   the number of clusters  $C_k$ , the number of GCN layers  $L$ .
3: // Training process:
4: repeat
5:   For  $b$  from 1 to  $|\mathcal{D}_{train}|/BatchSize$ :
6:     Initialize prediction loss  $\mathcal{L}^P \leftarrow 0$ 
7:     For each  $u \in \mathcal{D}_{train_b}$ :
8:        $\{G_{u,i}\}_{i=1}^n, \mathcal{I}_u \leftarrow \text{CodeGraphConstruction}(l_u)$ .
9:        $\mathcal{I}_u^*, \{G_{u,i}\}_{i=1}^n \leftarrow \text{UnwantedSignalsIdentification}(\mathcal{I}_u, \{S_{q_j}\}_{j=1}^M, \{G_{u,i}\}_{i=1}^n)$ .
10:       $\{(C_{u,cl}, \mathcal{W}_{u,cl})\}_{cl=1}^{C_k} \leftarrow \text{ClusterAwareWeakSignalFusion}(\{G_{u,i}\}_{i=1}^n, \{\vec{x}_{u,i}\}_{i=1}^{T_u}, C_k, L)$ .
11:      // Tuning Adaptor for PKT
12:      For  $t$  from 1 to  $T_u - 1$ :
13:        Construct prompt  $\vec{p}_{u,t}$  via  $\mathcal{I}_u, \{(C_{u,cl}, \mathcal{W}_{u,cl})\}_{cl=1}^{C_k}$ .
14:        // Calculate correction signal.
15:         $\vec{h}'_{t,u,a} \leftarrow (\mathbf{W}_A^T \cdot \mathbf{W}_B) \odot \vec{p}_{u,t}$ .
16:        // Update knowledge state  $\vec{h}_{u,t}$  learned by the PKT backbone  $\mathcal{M}_{PKT}$ .
17:         $\vec{h}'_{u,t} \leftarrow \vec{h}_{u,t} + \vec{h}'_{t,u,a}$ .
18:        // Compute programming performance at the step  $t + 1$  via programming records of  $u$  from step 1 to  $t$ .
19:         $\hat{r}_{u,t+1} = \mathcal{M}_{PKT}(l_{u,1:t}; \vec{h}'_{u,t})$ 
20:        Compute loss  $\mathcal{L}_P = \mathcal{L}_{u,t}^{pkt} + \mathcal{L}_{u,t}^{adaptor}$ .
21:      End for
22:      End for
23:      Calculate  $\theta_b \leftarrow \theta_{b-1} - lr \cdot \nabla_{\theta} \mathcal{L}^P$ , without gradient back-propagating.
24:      Calculate  $\mathcal{L}_b^{nav}$  via  $\theta_b, \theta_{b-1}$ .
25:       $\mathcal{L}^{Coda} \leftarrow \mathcal{L}_P + \mathcal{L}_b^{nav}$ 
26:      Update parameters  $\theta_b \leftarrow \theta_{b-1} - lr \cdot \nabla_{\theta} \mathcal{L}_b^{nav}$ , with gradient backpropagating.
27:    End for
28:  until epoch equals TotalEpoch
29:  // Test process:
30:  For each  $u \in \mathcal{D}_{test}$ :
31:    For  $t$  from 1 to  $T_u - 1$ :
32:       $\{G_{u,i}\}_{i=1}^n, \mathcal{I}_u \leftarrow \text{CodeGraphConstruction}(l_{u,1:t})$ .
33:       $\mathcal{I}_u^*, \{G_{u,i}\}_{i=1}^n \leftarrow \text{UnwantedSignalsIdentification}(\mathcal{I}_u, \{S_{q_j}\}_{j=1}^M, \{G_{u,i}\}_{i=1}^n)$ .
34:       $\{(C_{u,cl}, \mathcal{W}_{u,cl})\}_{cl=1}^{C_k} \leftarrow \text{ClusterAwareWeakSignalFusion}(\{G_{u,i}\}_{i=1}^n, \{\vec{x}_{u,i}\}_{i=1}^{T_u}, C_k, L)$ .
35:      Construct prompt  $\vec{p}_{u,t}$  via  $\mathcal{I}_u, \{(C_{u,cl}, \mathcal{W}_{u,cl})\}_{cl=1}^{C_k}$ .
36:       $\vec{h}'_{t,u,a} \leftarrow (\mathbf{W}_A^T \cdot \mathbf{W}_B) \odot \vec{p}_{u,t}$ .
37:       $\vec{h}'_{u,t} \leftarrow \vec{h}_{u,t} + \vec{h}'_{t,u,a}$ .
38:       $\hat{r}_{u,t+1} = \mathcal{M}_{PKT}(l_{u,1:t}; \vec{h}'_{u,t})$ 
39:      Compute performance metric using  $r_{u,t+1}$  and  $\hat{r}_{u,t+1}$ .
40:    End for
41:  End for

```

the CodeNet project[32]. We retain submission records in *C*, *C++*, and *Python* as the coding languages. Furthermore, since PST [21] only supports single-language programming scenarios, we additionally sample submissions written in *C++* from BePKT to generate the **BePKT_C++** dataset and submissions written in *C* from AtCoder to generate the **AtCoder_C** dataset, enabling us to evaluate PST. For each dataset, we exclude learners with fewer than five submissions to ensure that every learner has a sufficient number of records for tracing.

Algorithm 2 The Algorithm of Coda (Part 1)

```

1: Input:  $N$  learners' programming sequences  $\{l_1, l_2, \dots, l_N\}$ , the optimized backbone PKT model  $\mathcal{M}_{PKT}$ , correct solution feature set  $\{S_{q_j}\}_{j=1}^M$  of  $M$  questions, training set  $\mathcal{D}_{train}$ , test set  $\mathcal{D}_{test}$ .
2: Parameters ( $\theta$ ):  $\mathbf{W}, \mathbf{W}_a, \mathbf{W}_c, \mathbf{W}_A, \mathbf{W}_B$ .
3: Function  $\text{CodeGraphConstruction}(l_u)$ :
4:   // Construct code graph based on code semantic similarity.
5:   //  $l_u = \{(q_{u,1}, c_{u,1}, x_{u,1}, r_{u,1}), \dots, (q_{u,T_u}, c_{u,T_u}, x_{u,T_u}, r_{u,T_u})\}$ 
6:   For each pair of codes  $\langle x_{u,i}, x_{u,j} \rangle$  in  $l_u$ :
7:     Calculate indicator matrix  $M_{u,\langle i,j \rangle}$ .
8:   End for
9:   Generate binary adjacency matrix  $A_u$  via  $\text{Rank}_\epsilon$ .
10:  // Note that  $A_u$  may not guarantee a connected graph, and it may contain  $n$  subgraphs.
11:  Return code graph set:  $\{G_{u,i}\}_{i=1}^n$ , isolated code set:  $\mathcal{I}_u$ .
12: Function  $\text{UnwantedSignalsIdentification}(\mathcal{I}_u, \{S_{q_j}\}_{j=1}^M, \{G_{u,i}\}_{i=1}^n)$ :
13:  // Identify unwanted submissions from isolated nodes.
14:  Initialize an unwanted set  $\mathcal{I}_u^* \leftarrow \emptyset$ 
15:  For each isolated node  $i \in \mathcal{I}_u$ :
16:    Obtain question ID  $q_{u,i}$  corresponding to code  $i$ 
17:    For each  $x_{u,j'}$  in  $S_{q_{u,i}}$ :
18:       $s_{i,j'} \leftarrow \text{sim}(\mathbf{W} \odot \vec{x}_{u,i}, \mathbf{W} \odot \vec{x}_{u,j'})$ .
19:    End for
20:     $S_i \leftarrow \{s_{i,j'}; j' \in S_{q_{u,i}}\}$ 
21:    If  $\text{mean}(S_i) < \text{median}(S_i)$ :
22:      Mark node  $i$  as unwanted and  $\mathcal{I}_u^* \cup \{i\}$ .
23:    Else:
24:      Link code node  $i$  with  $\{G_{u,i}\}_{i=1}^n$ .
25:  End for
26:  Return unwanted set:  $\mathcal{I}_u^*$ , updated code graphs:  $\{G_{u,i}\}_{i=1}^n$ .
27: Function  $\text{ClusterAwareWeakSignalFusion}(\{G_{u,i}\}_{i=1}^n, \{\vec{x}_{u,i}\}_{i=1}^{T_u}, C_k, L)$ :
28:  // Apply  $L$ -layer cluster-aware GCN to update node features.
29:   $\{\vec{x}'_{u,i}\}_{i=1}^{T_u} \leftarrow \text{GCN}(\{\vec{x}_{u,i}\}_{i=1}^{T_u}, \{G_{u,i}\}_{i=1}^n, \mathbf{W}, \mathbf{W}_a, \mathbf{W}_c)$ 
30:   $C_k$  clusters  $\leftarrow \text{KMeans}(\{\vec{x}'_{u,i}\}, C_k)$ .
31:  Mark core submissions and weak signals for each cluster.
32:  Return a core submission and its related weak signal set for each cluster  $cl$ :  $\{(C_{u,cl}, \mathcal{W}_{u,cl})\}_{cl=1}^{C_k}$ .

```

C Algorithms

Algorithm 1 shows the pseudo-code of training and testing a PKT backbone model. Algorithms 2 and 3 show the workflow of Coda.